ФЕДЕРАЛЬНОЕ АГЕНТСТВО ПО ОБРАЗОВАНИЮ

ГОСУДАРСТВЕННОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ПРОФЕССИОНАЛЬНОГО ОБРАЗОВАНИЯ «САМАРСКИЙ НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ имени академика С.П. КОРОЛЁВА»

КАФЕДРА «ТЕХНИЧЕСКАЯ КИБЕРНЕТИКА»

ОТЧЕТ

ПО ЛАБОРАТОРНОЙ РАБОТЕ

«Объектно-ориентированное программирование»

ЛАБОРАТОРНАЯ РАБОТА №3

Студент Богданчиков М.А.

Группа 6301-030301D

Руководитель Борисов Д. С.

Оценка _____

САМАРА 2025

Цель лабораторной работы: Дополнить пакет для работы с функциями одной переменной, заданными в табличной форме, добавив классы исключений, новый класс функций и базовый интерфейс.

Ход выполнения лабораторной работы

Задание 1: Изучена документация:
java.lang.Exception, java.lang.IndexOutOfBoundsException, java.lang.ArrayIndexOutOfBoundsException, java.lang.IllegalArgumentException, java.lang.IllegalStateException

Задание 2: Созданы 2 класса исключения:
FunctionPointIndexOutOfBoundsException – исключение выхода за границы набора точек при обращении к ним по номеру, наследует от класса IndexOutOfBoundsException;

InappropriateFunctionPointException – исключение, выбрасываемое при попытке добавления или изменения точки функции несоответствующим образом, наследует от класса Exception.

```
package functions;

public class FunctionPointIndexOutOfBoundsException extends IndexOutOfBoundsException {
    public FunctionPointIndexOutOfBoundsException() {
        super();
    }

    public FunctionPointIndexOutOfBoundsException(String message) {
        super(message);
    }
}
```

Код 1.1

```
package functions;

public class InappropriateFunctionPointException extends Exception {
    public InappropriateFunctionPointException() {
        super();
    }

    public InappropriateFunctionPointException(String message) {
        super(message);
    }
}
```

Код 1.2

Задание 3:
Модифицирован класс табулированной функции TabulatedFunction, написанный в прошлой лабораторной работе. Теперь конструкторы и методы выбрасывают следующие исключения:

1.Оба конструктора класса выбрасывают
исключение IllegalArgumentException, если левая граница области определения больше или равна правой, а также если предлагаемое количество точек меньше двух.

2.Методы getPoint(), setPoint(), getPointX(), setPointX(), getPointY(), setPointY( ) и deletePoint() выбрасывают
исключение FunctionPointIndexOutOfBoundsException, если переданный в метод номер выходит за границы набора точек.

3. Методы setPoint() и setPointX() выбрасывают
исключение InappropriateFunctionPointException в том случае, если координата x задаваемой точки лежит вне интервала, определяемого значениями соседних точек табулированной функции.

4. Метод addPoint() выбрасывает
исключение InappropriateFunctionPointException, если в наборе точек функции есть точка, абсцисса которой совпадает с абсциссой добавляемой точки.

5. Метод deletePoint() выбрасывает исключение IllegalStateException, если на момент удаления точки количество точек в наборе менее трех.

```java
public ArrayTabulatedFunction(double leftX, double rightX, int pointsCount) {

    if (pointsCount < 2) {
        throw new IllegalArgumentException("Points count must be at least 2, got: " + pointsCount);
    }
    if (rightX - leftX < EPSILON) {
        throw new IllegalArgumentException(
                "Left border must be less than right border. Got: leftX=" + leftX + ", rightX=" + rightX
        );
    }

    this.pointsCount = pointsCount;
    this.points = new FunctionPoint[pointsCount + 10];

    double step = (rightX - leftX) / (pointsCount - 1);
    for (int i = 0; i < pointsCount; i++) {
        double x = leftX + i * step;
        points[i] = new FunctionPoint(x, 0.0);
    }
}

public ArrayTabulatedFunction(double leftX, double rightX, double[] values) {
    if (values.length < 2) {
        throw new IllegalArgumentException("Values array must have at least 2 elements");
    }
    if (rightX - leftX < EPSILON) {
        throw new IllegalArgumentException(
                "Left border must be less than right border. Got: leftX=" + leftX + ", rightX=" + rightX
        );
    }

    this.pointsCount = values.length;
    this.points = new FunctionPoint[pointsCount + 10];
```

Код 2.1

```java
public FunctionPoint getPoint(int index) {
    checkIndex(index);
    return new FunctionPoint(points[index]);
}

public void setPoint(int index, FunctionPoint point) throws InappropriateFunctionPointException {
    checkIndex(index);

    double newX = point.getX();
    double leftBound = (index > 0) ? points[index - 1].getX() : -Double.MAX_VALUE;
    double rightBound = (index < pointsCount - 1) ? points[index + 1].getX() : Double.MAX_VALUE;

    if (newX <= leftBound + EPSILON || newX >= rightBound - EPSILON) {
        throw new InappropriateFunctionPointException(
                "New X coordinate " + newX + " would break point ordering. " +
                        "Must be in (" + leftBound + ", " + rightBound + ")"
        );
    }

    points[index] = new FunctionPoint(point);
}

public double getPointX(int index) {
    checkIndex(index);
    return points[index].getX();
}
```

Код 2.2

```
public void setPointX(int index, double x) throws InappropriateFunctionPointException {
    checkIndex(index);

    double leftBound = (index > 0) ? points[index - 1].getX() : -Double.MAX_VALUE;
    double rightBound = (index < pointsCount - 1) ? points[index + 1].getX() : Double.MAX_VALUE;

    if (x <= leftBound + EPSILON || x >= rightBound - EPSILON) {
        throw new InappropriateFunctionPointException(
                "New X coordinate " + x + " would break point ordering. " +
                        "Must be in (" + leftBound + ", " + rightBound + ")"
        );
    }

    points[index].setX(x);
}

public double getPointY(int index) {
    checkIndex(index);
    return points[index].getY();
}

public void setPointY(int index, double y) {
    checkIndex(index);
    points[index].setY(y);
}
```

Код 2.3

```
public void deletePoint(int index) {
    if (pointsCount <= 2) {
        throw new IllegalStateException(
                "Cannot delete point: function must have at least 2 points. Current: " + pointsCount
        );
    }

    checkIndex(index);

    System.arraycopy(points, index + 1, points, index, pointsCount - index - 1);
    pointsCount--;
}

public void addPoint(FunctionPoint point) throws InappropriateFunctionPointException {

    for (int i = 0; i < pointsCount; i++) {
        if (Math.abs(points[i].getX() - point.getX()) < EPSILON) {
            throw new InappropriateFunctionPointException(
                    "Point with X = " + point.getX() + " already exists at index " + i
            );
        }
    }
    if (pointsCount == points.length) {
        FunctionPoint[] newArray = new FunctionPoint[points.length * 2];
        System.arraycopy(points, 0, newArray, 0, pointsCount);
```

Код 2.4

```
private void checkIndex(int index) {
    if (index < 0 || index >= pointsCount) {
        throw new FunctionPointIndexOutOfBoundsException(
                "Index " + index + " is out of bounds [0, " + (pointsCount - 1) + "]"
        );
    }
}
```

Код 2.5 (вспомогательный метод)

Задание 4: В пакете functions создан класс LinkedListTabulatedFunction, объект которого также описывает табулированную функцию, но для хранения набора точек в нем используется не массив, а связный список. Этот класс совмещает в себе две функции: с одной стороны, он описывает связный список и работу с ним, а с другой стороны, он описывает работу с табулированной функцией и ее точками. Для реализации первой функции реализованы:
FunctionNode (класс элементов списка, содержащий информационное поле для хранения данных типа FunctionPoint, а также поля для хранения ссылок на предыдущий и следующий элемент), метод getNodeByIndex (возвращающий ссылку на объект элемента списка по его номеру), метод addNodeToTail (добавляющий новый элемент в конец списка и возвращающий ссылку на объект этого элемента), метод addNodeByIndex (добавляющий новый элемент в указанную позицию списка и возвращающий ссылку на объект этого элемента), метод deleteNodeByIndex (удаляющий элемент списка по номеру и возвращающий ссылку на объект удаленного элемента).

```java
package functions;

public class LinkedListTabulatedFunction {
    protected class FunctionNode {
        FunctionPoint point;
        FunctionNode prev;
        FunctionNode next;

        FunctionNode(FunctionPoint point, FunctionNode prev, FunctionNode next) {
            this.point = point;
            this.prev = prev;
            this.next = next;
        }

        FunctionNode() {
            this.point = null;
            this.prev = this;
            this.next = this;
        }
    }

    private FunctionNode head;
    private int pointsCount;
    private FunctionNode lastAccessedNode;
    private int lastAccessedIndex;

    public LinkedListTabulatedFunction() {
        head = new FunctionNode();
        head.prev = head;
        head.next = head;
        pointsCount = 0;
        lastAccessedNode = head;
        lastAccessedIndex = -1;
    }
```

Код 3.1

```java
private FunctionNode getNodeByIndex(int index) {
    if (index < 0 || index >= pointsCount) {
        throw new FunctionPointIndexOutOfBoundsException();
    }


    int startIndex;
    FunctionNode current;

    if (Math.abs(index - lastAccessedIndex) < Math.abs(index)) {
        startIndex = lastAccessedIndex;
        current = lastAccessedNode;
    } else {
        startIndex = 0;
        current = head.next;
    }

    if (index > startIndex) {
        for (int i = startIndex; i < index; i++) {
            current = current.next;
        }
    } else {
        for (int i = startIndex; i > index; i--) {
            current = current.prev;
        }
    }

    lastAccessedNode = current;
    lastAccessedIndex = index;
    return current;
}
```

Код 3.2

```
private FunctionNode addNodeToTail() {
    FunctionNode newNode = new FunctionNode(new FunctionPoint(), head.prev, head);
    head.prev.next = newNode;
    head.prev = newNode;
    pointsCount++;
    return newNode;
}

private FunctionNode addNodeByIndex(int index) {
    FunctionNode nodeAtIndex = getNodeByIndex(index);
    FunctionNode newNode = new FunctionNode(new FunctionPoint(), nodeAtIndex.prev, nodeAtIndex);
    nodeAtIndex.prev.next = newNode;
    nodeAtIndex.prev = newNode;
    pointsCount++;
    lastAccessedNode = newNode;
    lastAccessedIndex = index;
    return newNode;
}
```

Код 3.3

```
private FunctionNode deleteNodeByIndex(int index) {
    if (pointsCount <= 2) {
        throw new IllegalStateException("Cannot delete point - minimum 2 points required");
    }

    FunctionNode nodeToDelete = getNodeByIndex(index);
    nodeToDelete.prev.next = nodeToDelete.next;
    nodeToDelete.next.prev = nodeToDelete.prev;
    pointsCount--;

    if (lastAccessedNode == nodeToDelete) {
        lastAccessedNode = nodeToDelete.next;
        if (lastAccessedNode == head) {
            lastAccessedNode = head.next;
            lastAccessedIndex = 0;
        }
    } else if (lastAccessedIndex > index) {
        lastAccessedIndex--;
    }

    return nodeToDelete;
}
```

Код 3.4

Задание 5: Для обеспечения второй функции
класса LinkedListTabulatedFunction реализованы в классе конструкторы и методы, аналогичные конструкторам и методам класса TabulatedFunction

```
public LinkedListTabulatedFunction(double leftX, double rightX, int pointsCount) {
    if (leftX >= rightX) {
        throw new IllegalArgumentException("Left border must be less than right border");
    }
    if (pointsCount < 2) {
        throw new IllegalArgumentException("Points count must be at least 2");
    }

    head = new FunctionNode();
    head.prev = head;
    head.next = head;
    pointsCount = 0;

    double step = (rightX - leftX) / (pointsCount - 1);
    for (int i = 0; i < pointsCount; i++) {
        double x = leftX + i * step;
        addPoint(new FunctionPoint(x, 0.0));
    }
}
```

Код 4.1

```
public double getPointX(int index) {
    return points[index].getX();
}

public void setPointX(int index, double x) {
    if (index < 0 || index >= pointsCount) {
        return;
    }

    double leftBound = (index > 0) ? points[index - 1].getX() : -Double.MAX_VALUE;
    double rightBound = (index < pointsCount - 1) ? points[index + 1].getX() : Double.MAX_VALUE;

    if (x > leftBound + EPSILON && x < rightBound - EPSILON) {
        points[index].setX(x);
    }
}

public double getPointY(int index) {
    return points[index].getY();
}

public void setPointY(int index, double y) {
    if (index >= 0 && index < pointsCount) {
        points[index].setY(y);
    }
}
```

Код 4.2

```java
public LinkedListTabulatedFunction(double leftX, double rightX, double[] values) {
    if (leftX >= rightX) {
        throw new IllegalArgumentException("Left border must be less than right border");
    }
    if (values.length < 2) {
        throw new IllegalArgumentException("Points count must be at least 2");
    }

    head = new FunctionNode();
    head.prev = head;
    head.next = head;
    pointsCount = 0;

    double step = (rightX - leftX) / (values.length - 1);
    for (int i = 0; i < values.length; i++) {
        double x = leftX + i * step;
        addPoint(new FunctionPoint(x, values[i]));
    }
}

public double getLeftDomainBorder() {
    if (pointsCount == 0) {
        throw new IllegalStateException("Function has no points");
    }
    return head.next.point.getX();
}
```

Код 4.3

```java
public double getRightDomainBorder() {
    if (pointsCount == 0) {
        throw new IllegalStateException("Function has no points");
    }
    return head.prev.point.getX();
}


public double getFunctionValue(double x) {
    if (x < getLeftDomainBorder() - EPSILON || x > getRightDomainBorder() + EPSILON) {
        return Double.NaN;
    }
    FunctionNode current = head.next;
    while (current != head) {
        if (Math.abs(current.point.getX() - x) < EPSILON) {
            return current.point.getY();
        }
        current = current.next;
    }


    current = head.next;
    while (current.next != head) {
        double x1 = current.point.getX();
        double x2 = current.next.point.getX();

        if (x >= x1 - EPSILON && x <= x2 + EPSILON) {
            double y1 = current.point.getY();
            double y2 = current.next.point.getY();
            return y1 + (y2 - y1) * (x - x1) / (x2 - x1);
        }
        current = current.next;
    }

    return Double.NaN;
}
```

Код 4.4

```
public int getPointsCount() {
    return pointsCount;
}

public double getLeftDomainBorder() {
    if (pointsCount == 0) {
        throw new IllegalStateException("Function has no points");
    }
    return head.next.point.getX();
}

public double getRightDomainBorder() {
    if (pointsCount == 0) {
        throw new IllegalStateException("Function has no points");
    }
    return head.prev.point.getX();
}
```

Код 4.5

```
public double getFunctionValue(double x) {

    if (x < getLeftDomainBorder() - EPSILON || x > getRightDomainBorder() + EPSILON) {
        return Double.NaN;
    }

    FunctionNode current = head.next;
    while (current != head) {
        if (Math.abs(current.point.getX() - x) < EPSILON) {
            return current.point.getY();
        }
        current = current.next;
    }

    current = head.next;
    while (current.next != head) {
        double x1 = current.point.getX();
        double x2 = current.next.point.getX();

        if (x >= x1 - EPSILON && x <= x2 + EPSILON) {
            double y1 = current.point.getY();
            double y2 = current.next.point.getY();
            return y1 + (y2 - y1) * (x - x1) / (x2 - x1);
        }
        current = current.next;
    }

    return Double.NaN;
}
public int getPointsCount() {
    return pointsCount;
}
public FunctionPoint getPoint(int index) {
    checkIndex(index);

    return new FunctionPoint(getNodeByIndex(index).point);
}
```

Код 4.6

```java
public int getPointsCount() {
    return pointsCount;
}
public FunctionPoint getPoint(int index) {
    checkIndex(index);

    return new FunctionPoint(getNodeByIndex(index).point);
}
public void setPoint(int index, FunctionPoint point) throws InappropriateFunctionPointException {
    checkIndex(index);
    FunctionNode node = getNodeByIndex(index);

    double newX = point.getX();
    double leftBound = (index > 0) ? node.prev.point.getX() : -Double.MAX_VALUE;
    double rightBound = (index < pointsCount - 1) ? node.next.point.getX() : Double.MAX_VALUE;

    if (newX <= leftBound + EPSILON || newX >= rightBound - EPSILON) {
        throw new InappropriateFunctionPointException("X coordinate violates ordering");
    }

    FunctionNode current = head.next;
    while (current != head) {
        if (current != node && Math.abs(current.point.getX() - newX) < EPSILON) {
            throw new InappropriateFunctionPointException("Point with same X already exists");
        }
        current = current.next;
    }
    node.point = new FunctionPoint(point);
}
```

Код 4.7

```java
public double getPointX(int index) {
    checkIndex(index);
    return getNodeByIndex(index).point.getX();
}


public void setPointX(int index, double x) throws InappropriateFunctionPointException {
    checkIndex(index);
    FunctionNode node = getNodeByIndex(index);

    double leftBound = (index > 0) ? node.prev.point.getX() : -Double.MAX_VALUE;
    double rightBound = (index < pointsCount - 1) ? node.next.point.getX() : Double.MAX_VALUE;

    if (x <= leftBound + EPSILON || x >= rightBound - EPSILON) {
        throw new InappropriateFunctionPointException("X coordinate violates ordering");
    }

    FunctionNode current = head.next;
    while (current != head) {
        if (current != node && Math.abs(current.point.getX() - x) < EPSILON) {
            throw new InappropriateFunctionPointException("Point with same X already exists");
        }
        current = current.next;
    }

    node.point.setX(x);
}
```

Код 4.8

```java
public double getPointY(int index) {
    checkIndex(index);
    return getNodeByIndex(index).point.getY();
}

public void setPointY(int index, double y) {
    checkIndex(index);
    getNodeByIndex(index).point.setY(y);
}

private void checkIndex(int index) {
    if (index < 0 || index >= pointsCount) {
        throw new FunctionPointIndexOutOfBoundsException("Index: " + index);
    }
}
public void deletePoint(int index) {
    checkIndex(index);
    if (pointsCount <= 2) {
        throw new IllegalStateException("Cannot delete point - minimum 2 points required");
    }

    deleteNodeByIndex(index);
}
```

Код 4.9

```java
public void addPoint(FunctionPoint point) throws InappropriateFunctionPointException {

    FunctionNode current = head.next;
    while (current != head) {
        if (Math.abs(current.point.getX() - point.getX()) < EPSILON) {
            throw new InappropriateFunctionPointException("Point with same X already exists");
        }
        current = current.next;
    }
    int insertIndex = 0;
    current = head.next;
    while (current != head && current.point.getX() < point.getX()) {
        current = current.next;
        insertIndex++;
    }
    if (insertIndex == pointsCount) {
        FunctionNode newNode = new FunctionNode(new FunctionPoint(point), head.prev, head);
        head.prev.next = newNode;
        head.prev = newNode;
    } else {
        FunctionNode nodeAtIndex = getNodeByIndex(insertIndex);
        FunctionNode newNode = new FunctionNode(new FunctionPoint(point), nodeAtIndex.prev, nodeAtIndex);
        nodeAtIndex.prev.next = newNode;
        nodeAtIndex.prev = newNode;
    }
    pointsCount++;
    lastAccessedNode = head;
    lastAccessedIndex = -1;
}
```

Код 4.10

```
private FunctionNode lastInterpolationNode = head;
public double getFunctionValue(double x) {
    if (x < getLeftDomainBorder() - EPSILON || x > getRightDomainBorder() + EPSILON) {
        return Double.NaN;
    }
    FunctionNode current = lastInterpolationNode;
    while (current != head && current.point.getX() > x + EPSILON) {
        current = current.prev;
    }
    while (current.next != head && current.next.point.getX() < x - EPSILON) {
        current = current.next;
    }
    if (Math.abs(current.point.getX() - x) < EPSILON) {
        lastInterpolationNode = current;
        return current.point.getY();
    }
    if (current.next != head && Math.abs(current.next.point.getX() - x) < EPSILON) {
        lastInterpolationNode = current.next;
        return current.next.point.getY();
    }
    if (current.next != head &&
            x >= current.point.getX() - EPSILON &&
            x <= current.next.point.getX() + EPSILON) {

        double x1 = current.point.getX();
        double x2 = current.next.point.getX();
        double y1 = current.point.getY();
        double y2 = current.next.point.getY();
        lastInterpolationNode = current;
        return y1 + (y2 - y1) * (x - x1) / (x2 - x1);
    }
    return Double.NaN;
}
```

Код 4.11

Задание 6: Класс TabulatedFunction переименован в класс ArrayTabulatedFunction. Создан интерфейс TabulatedFunction, содержащий объявления общих методов классов ArrayTabulatedFunction и LinkedListTabulatedFunction, оба класса функций реализовывают интерфейс.

```java
package functions;

public interface TabulatedFunction {
    double getLeftDomainBorder();
    double getRightDomainBorder();
    double getFunctionValue(double x);
    int getPointsCount();
    FunctionPoint getPoint(int index);
    void setPoint(int index, FunctionPoint point) throws InappropriateFunctionPointException;
    double getPointX(int index);
    void setPointX(int index, double x) throws InappropriateFunctionPointException;
    double getPointY(int index);
    void setPointY(int index, double y);
    void deletePoint(int index);
    void addPoint(FunctionPoint point) throws InappropriateFunctionPointException;
    String toString();
}
```

Код 5.1

```java
public class ArrayTabulatedFunction implements TabulatedFunction{
```

Код 5.2

```java
public class LinkedListTabulatedFunction implements TabulatedFunction{
```

Код 5.3

Задание 7: создан класс Main, с помощью которого будут проверяться работа написанных классов, и выведен результат его выполнения в консоль.

```java
import functions.*;
public class Main {
    public static void main(String[] args) {
        try {
            TabulatedFunction arrayFunc = new ArrayTabulatedFunction(0, 10, 5);
            testFunction(arrayFunc);
            TabulatedFunction listFunc = new LinkedListTabulatedFunction(0, 10, 5);
            testFunction(listFunc);
            testExceptions();

        } catch (Exception e) {
            System.out.println("Exception: " + e.getClass().getName() + " - " + e.getMessage());
        }
    }

    private static void testFunction(TabulatedFunction func) {
        System.out.println("Testing function:");
        System.out.println("Left border: " + func.getLeftDomainBorder());
        System.out.println("Right border: " + func.getRightDomainBorder());
        System.out.println("Points count: " + func.getPointsCount());

        double testX = 5.0;
        System.out.println("Value at x=" + testX + ": " + func.getFunctionValue(testX));

        System.out.println("Value at x=-1: " + func.getFunctionValue(-1));
        System.out.println("Value at x=11: " + func.getFunctionValue(11));
    }
```

Код 6.1

```java
private static void testExceptions() {
    try {
        TabulatedFunction func = new ArrayTabulatedFunction(10, 0, 5);
    } catch (IllegalArgumentException e) {
        System.out.println("Caught IllegalArgumentException: " + e.getMessage());
    }

    try {
        TabulatedFunction func = new ArrayTabulatedFunction(0, 10, 1);
    } catch (IllegalArgumentException e) {
        System.out.println("Caught IllegalArgumentException: " + e.getMessage());
    }

    try {
        TabulatedFunction func = new ArrayTabulatedFunction(0, 10, 3);
        func.getPoint(5);
    } catch (FunctionPointIndexOutOfBoundsException e) {
        System.out.println("Caught FunctionPointIndexOutOfBoundsException: " + e.getMessage());
    }

    try {
        TabulatedFunction func = new ArrayTabulatedFunction(0, 10, 3);
        func.deletePoint(1);
        func.deletePoint(0);
    } catch (IllegalStateException e) {
        System.out.println("Caught IllegalStateException: " + e.getMessage());
    }
}
```

Код 6.2

```
PS C:\Javalabs\lab3> javac functions/*.java
PS C:\Javalabs\lab3> javac Main.java
PS C:\Javalabs\lab3> java Main
Testing function:
Left border: 0.0
Right border: 10.0
Points count: 5
Value at x=5.0: 0.0
Value at x=-1: NaN
Value at x=11: NaN
Testing function:
Left border: 0.0
Right border: 10.0
Points count: 5
Value at x=5.0: 0.0
Value at x=-1: NaN
Value at x=11: NaN
Caught IllegalArgumentException: Left border must be less than right border. Got: leftX=10.0, rightX=0.0
Caught IllegalArgumentException: Points count must be at least 2, got: 1
Caught FunctionPointIndexOutOfBoundsException: Index 5 is out of bounds [0, 2]
Caught IllegalStateException: Cannot delete point: function must have at least 2 points. Current: 2
PS C:\Javalabs\lab3>
```

Реализация 6